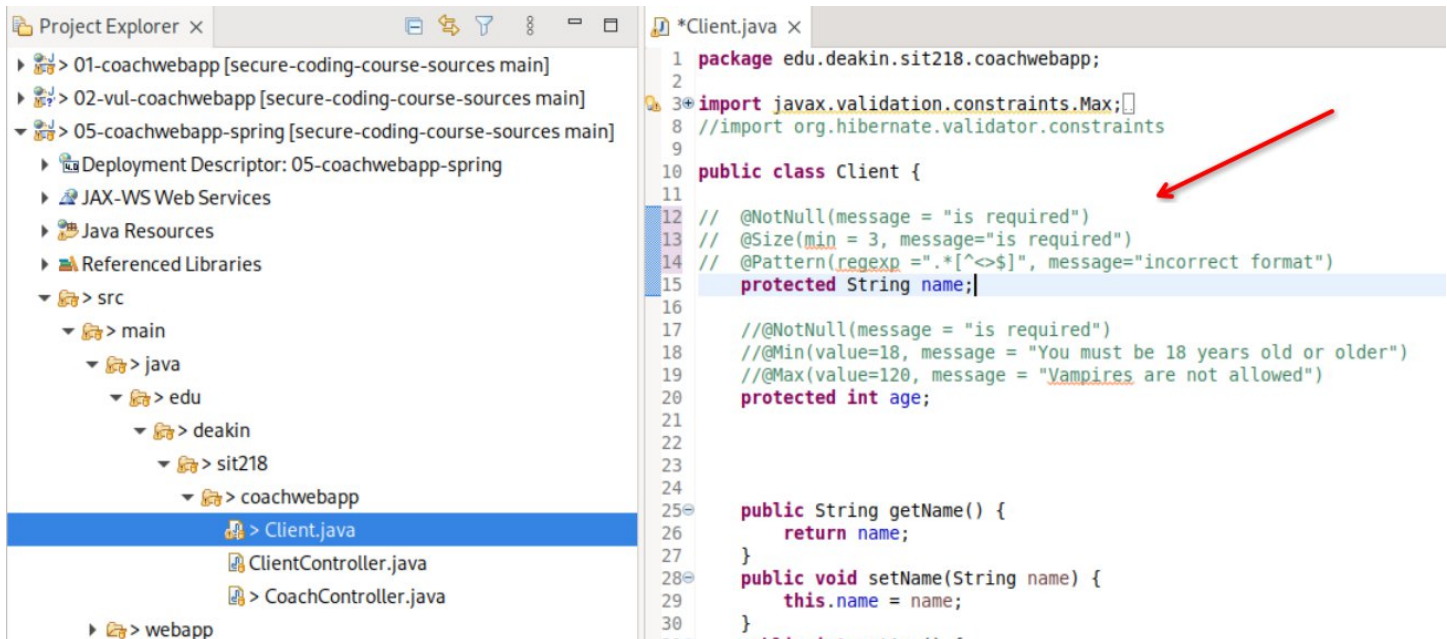


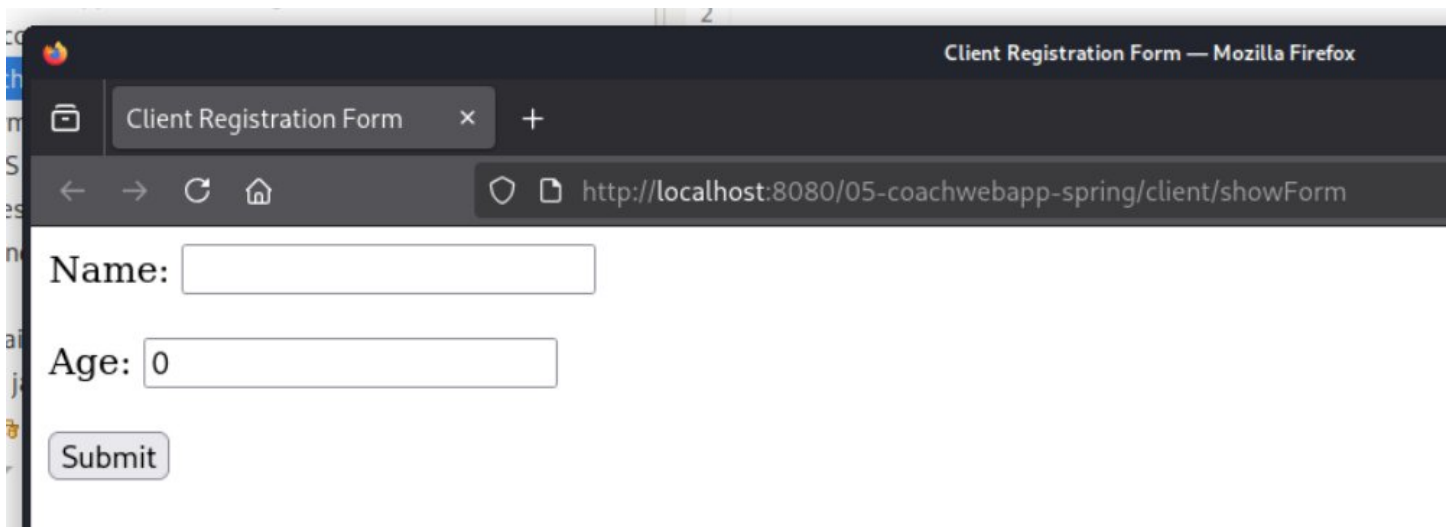
Task 4.1P

I commented those lines as instructed.



```
1 package edu.deakin.sit218.coachwebapp;
2
3 import javax.validation.constraints.Max;
4 //import org.hibernate.validator.constraints
5
6 public class Client {
7
8     // @NotNull(message = "is required")
9     // @Size(min = 3, message="is required")
10    // @Pattern(regexp = ".*[<>]", message="incorrect format")
11    protected String name;
12
13    // @NotNull(message = "is required")
14    // @Min(value=18, message = "You must be 18 years old or older")
15    // @Max(value=120, message = "Vampires are not allowed")
16    protected int age;
17
18    public String getName() {
19        return name;
20    }
21    public void setName(String name) {
22        this.name = name;
23    }
24}
```

Then I started the web server.



Client Registration Form — Mozilla Firefox

Client Registration Form

http://localhost:8080/05-coachwebapp-spring/client/showForm

Name:

Age:

I then set the characters list up.

```
(kali@kali)-[~/Desktop/task41p]
$ mkdir wordlist

(kali@kali)-[~/Desktop/task41p]
$ cd wordlist

(kali@kali)-[~/Desktop/task41p/wordlist]
$ wget https://raw.githubusercontent.com/xmendez/wfuzz/refs/heads/master/wordlist/stress/alphanum_case.txt
--2026-08-20 12:07:31-- https://raw.githubusercontent.com/xmendez/wfuzz/refs/heads/master/wordlist/stress/alphanum_case.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 124 [text/plain]
Saving to: 'alphanum_case.txt'

alphanum_case.txt      100%[=====>]      124  --.-KB/s   in 0s

2026-08-20 12:07:32 (2.67 MB/s) - 'alphanum_case.txt' saved [124/124]

(kali@kali)-[~/Desktop/task41p/wordlist]
$ mv alphanum_case.txt chars.txt

(kali@kali)-[~/Desktop/task41p/wordlist]
$ head -n 5 chars.txt
0
1
2
3
4
```

I then fuzzed the name. All characters here returns a 200 (accepted) because `name` has no validation mechanisms in place.

```
(kali@kali)-[~/Desktop/task41p/wordlist]
$ wfuzz -c -w chars.txt "http://localhost:8080/05-coachwebapp-spring/workout/processForm?name=FUZZ"
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer *
*****

Target: http://localhost:8080/05-coachwebapp-spring/workout/processForm?name=FUZZ
Total requests: 62



| ID         | Response | Lines | Word | Chars  | Payload |
|------------|----------|-------|------|--------|---------|
| 000000046: | 200      | 15 L  | 31 W | 229 Ch | "J"     |
| 000000050: | 200      | 15 L  | 31 W | 229 Ch | "N"     |
| 000000049: | 200      | 15 L  | 31 W | 229 Ch | "M"     |
| 000000007: | 200      | 15 L  | 31 W | 229 Ch | "6"     |
| 000000001: | 200      | 15 L  | 31 W | 229 Ch | "0"     |
| 000000031: | 200      | 15 L  | 31 W | 229 Ch | "u"     |
| 000000015: | 200      | 15 L  | 31 W | 229 Ch | "e"     |
| 000000048: | 200      | 15 L  | 31 W | 229 Ch | "L"     |
| 000000003: | 200      | 15 L  | 31 W | 229 Ch | "2"     |
| 000000047: | 200      | 15 L  | 31 W | 229 Ch | "K"     |
| 000000044: | 200      | 15 L  | 31 W | 229 Ch | "H"     |
| 000000042: | 200      | 15 L  | 31 W | 229 Ch | "F"     |
| 000000041: | 200      | 15 L  | 31 W | 229 Ch | "E"     |
| 000000038: | 200      | 15 L  | 31 W | 229 Ch | "B"     |
| 000000039: | 200      | 15 L  | 31 W | 229 Ch | "C"     |
| 000000036: | 200      | 15 L  | 31 W | 229 Ch | "z"     |
| 000000043: | 200      | 15 L  | 31 W | 229 Ch | "G"     |
| 000000058: | 200      | 15 L  | 31 W | 229 Ch | "V"     |
| 000000052: | 200      | 15 L  | 31 W | 229 Ch | "P"     |
| 000000061: | 200      | 15 L  | 31 W | 229 Ch | "Y"     |
| 000000062: | 200      | 15 L  | 31 W | 229 Ch | "Z"     |
| 000000054: | 200      | 15 L  | 31 W | 229 Ch | "R"     |



Total time: 0.208090
Processed Requests: 62
Filtered Requests: 0
Requests/sec.: 297.9468
```

I then fuzzed the age.

```
(kali@kali)-[~/Desktop/task41p/wordlist]
$ wfuzz -c -w chars.txt "http://localhost:8080/05-coachwebapp-spring/workout/processForm?age=FUZZ"
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

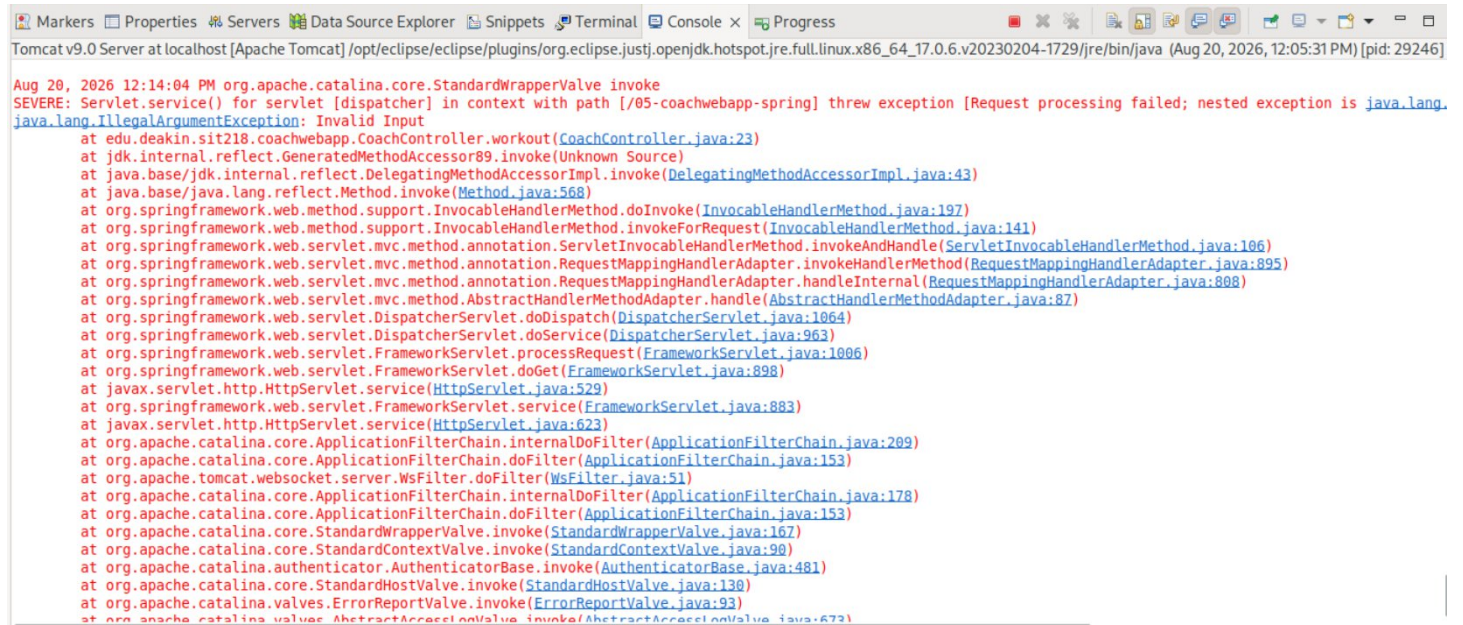
Target: http://localhost:8080/05-coachwebapp-spring/workout/processForm?age=FUZZ
Total requests: 62
```

ID	Response	Lines	Word	Chars	Payload
000000001:	200	15 L	31 W	232 Ch	"0"
000000048:	404	0 L	0 W	0 Ch	"L"
000000015:	404	0 L	0 W	0 Ch	"e"
000000003:	200	15 L	31 W	232 Ch	"2"
000000007:	200	15 L	31 W	232 Ch	"6"
000000031:	404	0 L	0 W	0 Ch	"u"
000000040:	404	0 L	0 W	0 Ch	"D"
000000044:	404	0 L	0 W	0 Ch	"H"
000000046:	404	0 L	0 W	0 Ch	"J"
000000047:	404	0 L	0 W	0 Ch	"K"
000000041:	404	0 L	0 W	0 Ch	"E"
000000050:	404	0 L	0 W	0 Ch	"N"
000000045:	404	0 L	0 W	0 Ch	"I"
000000049:	404	0 L	0 W	0 Ch	"M"
000000034:	404	0 L	0 W	0 Ch	"x"
000000035:	404	0 L	0 W	0 Ch	"y"
000000043:	404	0 L	0 W	0 Ch	"G"
000000012:	404	0 L	0 W	0 Ch	"b"
000000010:	200	15 L	31 W	232 Ch	"9"
000000009:	200	15 L	31 W	232 Ch	"8"
000000006:	200	15 L	31 W	232 Ch	"5"
000000008:	200	15 L	31 W	232 Ch	"7"
000000028:	404	0 L	0 W	0 Ch	"r"
000000016:	404	0 L	0 W	0 Ch	"f"
000000025:	404	0 L	0 W	0 Ch	"o"
000000005:	200	15 L	31 W	232 Ch	"4"
000000004:	200	15 L	31 W	232 Ch	"3"
000000023:	404	0 L	0 W	0 Ch	"m"
000000002:	200	15 L	31 W	232 Ch	"1"
000000051:	404	0 L	0 W	0 Ch	"0"
000000057:	404	0 L	0 W	0 Ch	"U"
000000061:	404	0 L	0 W	0 Ch	"v"
000000026:	404	0 L	0 W	0 Ch	"p"
000000053:	404	0 L	0 W	0 Ch	"q"
000000013:	404	0 L	0 W	0 Ch	"c"
000000059:	404	0 L	0 W	0 Ch	"w"
000000060:	404	0 L	0 W	0 Ch	"X"
000000052:	404	0 L	0 W	0 Ch	"P"
000000056:	404	0 L	0 W	0 Ch	"T"
000000017:	404	0 L	0 W	0 Ch	"g"
000000062:	404	0 L	0 W	0 Ch	"Z"
000000011:	404	0 L	0 W	0 Ch	"a"
000000054:	404	0 L	0 W	0 Ch	"R"
000000058:	404	0 L	0 W	0 Ch	"V"
000000055:	404	0 L	0 W	0 Ch	"S"

```
Total time: 0
Processed Requests: 62
Filtered Requests: 0
Requests/sec.: 0
```

In the notes, it returns a status code of 500. However, when I tested it out, it returned a status code of 404 for alphabetical characters and returns 200s only for numeric characters. This could be because Tomcat sometimes falls back into 404 instead of showing the actual code like 500. However, the program's validation still works as expected with no sensitive information reveal.

Below is a screenshot of the error that I received from the console. This is the full traceback.

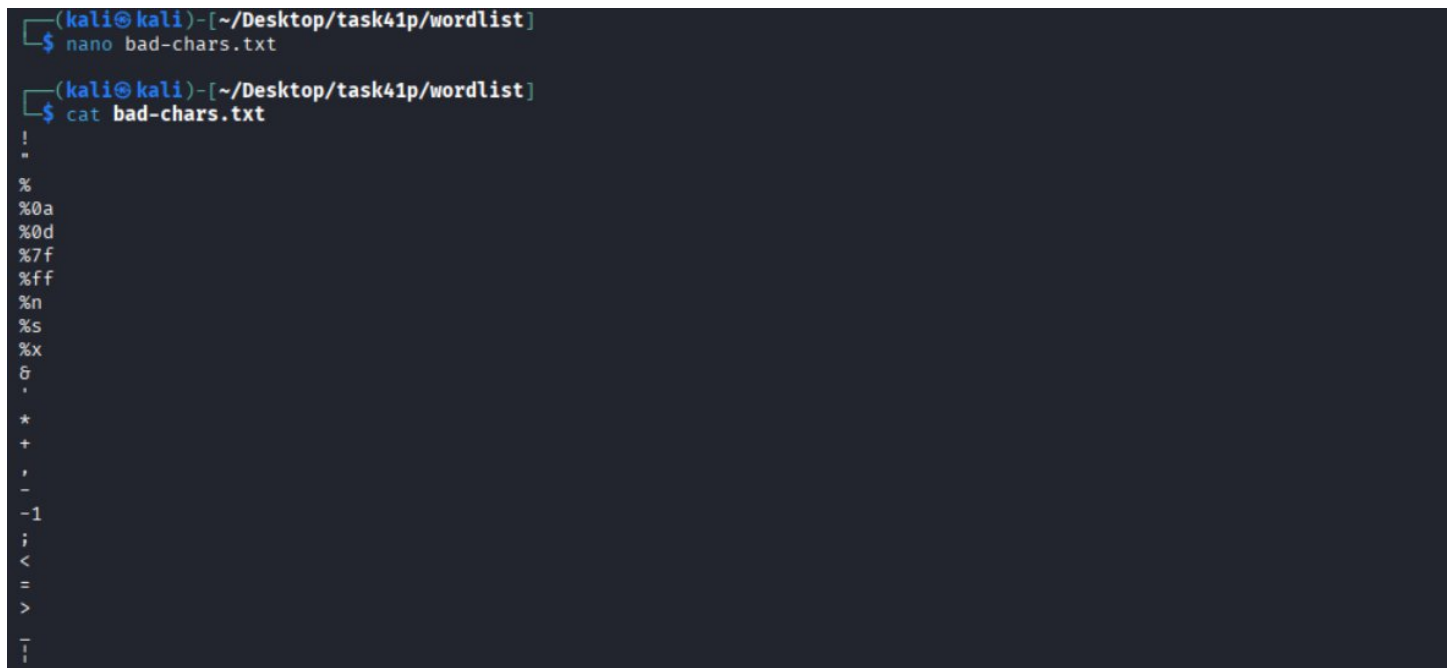


```
Tomcat v9.0 Server at localhost [Apache Tomcat] /opt/eclipse/eclipse/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.linux.x86_64_17.0.6.v20230204-1729/jre/bin/java (Aug 20, 2026, 12:05:31 PM) [pid: 29246]

Aug 20, 2026 12:14:04 PM org.apache.catalina.core.StandardWrapperValve invoke
SEVERE: Servlet.service() for servlet [dispatcher] in context with path [/05-coachwebapp-spring] threw exception [Request processing failed; nested exception is java.lang.
java.lang.IllegalArgumentException: Invalid Input
    at edu.deakin.sit218.coachwebapp.CoachController.workout(CoachController.java:23)
    at jdk.internal.reflect.GeneratedMethodAccessor89.invoke(Unknown Source)
    at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.base/java.lang.reflect.Method.invoke(Method.java:568)
    at org.springframework.web.method.support.InvocableHandlerMethod.doInvoke(InvocableHandlerMethod.java:197)
    at org.springframework.web.method.support.InvocableHandlerMethod.invokeForRequest(InvocableHandlerMethod.java:141)
    at org.springframework.web.servlet.mvc.method.annotation.ServletInvocableHandlerMethod.invokeAndHandle(ServletInvocableHandlerMethod.java:106)
    at org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.invokeHandlerMethod(RequestMappingHandlerAdapter.java:895)
    at org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.handleInternal(RequestMappingHandlerAdapter.java:808)
    at org.springframework.web.servlet.mvc.method.AbstractHandlerMethodAdapter.handle(AbstractHandlerMethodAdapter.java:87)
    at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:1064)
    at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:963)
    at org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1006)
    at org.springframework.web.servlet.FrameworkServlet.doGet(FrameworkServlet.java:898)
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:529)
    at org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:883)
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:623)
    at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:209)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:153)
    at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:51)
    at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:178)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:153)
    at org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:167)
    at org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:90)
    at org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:481)
    at org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:130)
    at org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:93)
    at org.apache.catalina.valves.AbstractAccessLogValve.invoke(AbstractAccessLogValve.java:672)
```

The `IllegalArgumentException` does make sense here as we store the age in an `int` datatype and it cannot store alphabetical characters.

I then made a file with all the bad characters as instructed.



```
(kali@kali) - [~/Desktop/task41p/wordlist]
$ nano bad-chars.txt

(kali@kali) - [~/Desktop/task41p/wordlist]
$ cat bad-chars.txt
!
"
%
%0a
%0d
%7f
%ff
%n
%s
%x
&
'
*
+
,
-
~
-1
;
<
=
>
_
`
~
```

Using this file, I fuzzed the name and age again.

```
(kali@kali)~/Desktop/task41p/wordlist
$ wfuzz -c -w bad-chars.txt "http://localhost:8080/05-coachwebapp-spring/workout/processForm?name=FUZZ"
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*
*****

Target: http://localhost:8080/05-coachwebapp-spring/workout/processForm?name=FUZZ
Total requests: 24
```

ID	Response	Lines	Word	Chars	Payload
000000022:	200	15 L	31 W	229 Ch	" "
000000020:	200	15 L	31 W	229 Ch	"="
000000007:	200	15 L	31 W	229 Ch	"%ff"
000000014:	200	15 L	30 W	229 Ch	"+"
000000015:	200	15 L	31 W	229 Ch	","
000000003:	200	15 L	31 W	232 Ch	"%"
000000001:	200	15 L	31 W	229 Ch	"!"
000000017:	200	15 L	31 W	230 Ch	"-1"
000000016:	200	15 L	31 W	229 Ch	"-"
000000021:	400	11 L	136 W	2109 Ch	
000000024:	200	15 L	30 W	228 Ch	"http://localhost:8080/05-coachwebapp-spring/workout/processForm?name="
000000018:	200	15 L	31 W	229 Ch	";"
000000019:	400	11 L	136 W	2109 Ch	"<"
000000006:	200	15 L	31 W	229 Ch	"%7f"
000000023:	400	11 L	136 W	2133 Ch	".."
000000012:	200	15 L	31 W	229 Ch	"."
000000008:	200	15 L	31 W	232 Ch	"%n"
000000004:	200	16 L	30 W	229 Ch	"%0a"
000000005:	200	15 L	30 W	229 Ch	"%0d"
000000002:	400	11 L	136 W	2113 Ch	""
000000013:	200	15 L	31 W	229 Ch	"*"
000000011:	200	15 L	30 W	228 Ch	"6"
000000010:	200	15 L	31 W	232 Ch	"%x"
000000009:	200	15 L	31 W	232 Ch	"%s"

```
Total time: 0.048255
Processed Requests: 24
Filtered Requests: 0
Requests/sec.: 497.3483
```

```
(kali@kali)~/Desktop/task41p/wordlist
$ wfuzz -c -w bad-chars.txt "http://localhost:8080/05-coachwebapp-spring/workout/processForm?age=FUZZ"
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*
*****

Target: http://localhost:8080/05-coachwebapp-spring/workout/processForm?age=FUZZ
Total requests: 24
```

ID	Response	Lines	Word	Chars	Payload
000000007:	404	0 L	0 W	0 Ch	"%ff"
000000021:	400	11 L	136 W	2107 Ch	
000000015:	404	0 L	0 W	0 Ch	","
000000023:	400	11 L	136 W	2131 Ch	".."
000000024:	404	0 L	0 W	0 Ch	"http://localhost:8080/05-coachwebapp-spring/workout/processForm?age="
000000020:	404	0 L	0 W	0 Ch	"="
000000014:	404	0 L	0 W	0 Ch	"+"
000000022:	404	0 L	0 W	0 Ch	" "
000000001:	404	0 L	0 W	0 Ch	"!"
000000003:	200	15 L	31 W	232 Ch	"%"
000000019:	400	11 L	136 W	2107 Ch	"<"
000000017:	200	15 L	31 W	232 Ch	"-1"
000000016:	404	0 L	0 W	0 Ch	"-"
000000013:	404	0 L	0 W	0 Ch	"*"
000000005:	404	0 L	0 W	0 Ch	"%0d"
000000009:	200	15 L	31 W	232 Ch	"%s"
000000012:	404	0 L	0 W	0 Ch	"."
000000008:	200	15 L	31 W	232 Ch	"%n"
000000011:	404	0 L	0 W	0 Ch	"6"
000000018:	404	0 L	0 W	0 Ch	""
000000002:	400	11 L	136 W	2111 Ch	""
000000006:	404	0 L	0 W	0 Ch	"%7f"
000000010:	200	15 L	31 W	232 Ch	"%x"
000000004:	404	0 L	0 W	0 Ch	"%0a"

```
Total time: 0
Processed Requests: 24
Filtered Requests: 0
Requests/sec.: 0
```

For the name field, most of the payloads are returning 200 because there is no proper validation added in Client.java yet. Some characters like <, >, .., and "" are returning 400, but this is mainly because Tomcat blocks them at the HTTP level before the request even reaches the application. Most non numeric values for age return 404 because the age is stored as an int, which cannot store letters or other non-numeric characters. Normal numeric values are working fine and returning 200.

I then added some basic validation and ensured that the development server was also updated accordingly.

```
Client.java x ClientController.java CoachController.java
import javax.validation.constraints.Max;
//import org.hibernate.validator.constraints
public class Client {
    @NotNull(message = "is required")
    @Size(min = 3, message="is required")
    // @Pattern(regex = ".*[^\>$]", message="incorrect format")
    protected String name;
```

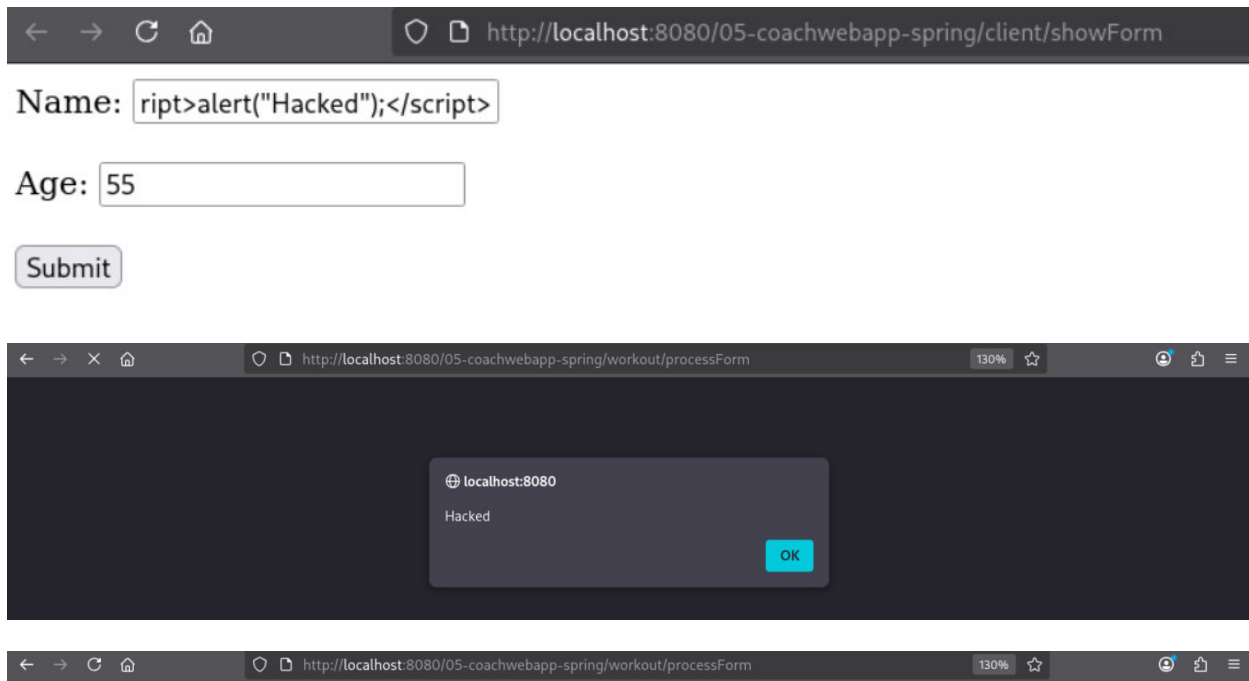
I then created the common wordlist and fuzzed the name with it.

```
(kali@kali)-[~/Desktop/task41p/wordlist]
$ nano common.txt
(kali@kali)-[~/Desktop/task41p/wordlist]
$ cat common.txt
@
00
01
02
03
1
10
100
1000
123
2
20
200
2000
2001
2002
2003
2004
2005
3
a
aa
aaa
certificates
certs
cfdocs
cfg
cgi
cgibin
```

```
000000001: 404 0 L 0 W 0 Ch "0"
000000007: 404 0 L 0 W 0 Ch "10"
000000026: 200 15 L 31 W 234 Ch "cfdocs"
000000028: 200 15 L 31 W 231 Ch "cgi"
000000015: 200 15 L 31 W 232 Ch "2001"
000000027: 200 15 L 31 W 231 Ch "cfg"
000000022: 404 0 L 0 W 0 Ch "aa"
000000024: 200 15 L 31 W 240 Ch "certificates"
000000025: 200 15 L 31 W 233 Ch "certs"
000000029: 200 15 L 31 W 234 Ch "cgibin"
000000003: 404 0 L 0 W 0 Ch "01"
000000023: 200 15 L 31 W 231 Ch "aaa"
000000021: 404 0 L 0 W 0 Ch "a"
000000020: 404 0 L 0 W 0 Ch "3"
000000011: 404 0 L 0 W 0 Ch "2"
000000012: 404 0 L 0 W 0 Ch "20"
000000013: 200 15 L 31 W 231 Ch "200"
000000018: 200 15 L 31 W 232 Ch "2004"
000000019: 200 15 L 31 W 232 Ch "2005"
000000016: 200 15 L 31 W 232 Ch "2002"
000000017: 200 15 L 31 W 232 Ch "2003"
000000030: 200 15 L 31 W 235 Ch "cgi-bin"
000000008: 200 15 L 31 W 231 Ch "100"
000000005: 404 0 L 0 W 0 Ch "03"
000000010: 200 15 L 31 W 231 Ch "123"
000000032: 200 15 L 31 W 232 Ch "chan"
000000006: 404 0 L 0 W 0 Ch "1"
000000009: 200 15 L 31 W 232 Ch "1000"
000000002: 404 0 L 0 W 0 Ch "00"
000000014: 200 15 L 31 W 232 Ch "2000"
000000004: 404 0 L 0 W 0 Ch "02"
000000041: 200 15 L 31 W 238 Ch "classified"
000000047: 200 15 L 31 W 231 Ch "cmd"
000000046: 404 0 L 0 W 0 Ch "cm"
000000044: 200 15 L 31 W 235 Ch "clients"
000000042: 200 15 L 31 W 239 Ch "classifieds"
000000048: 404 0 L 0 W 0 Ch "http://localhost:8080/05-coachwebapp-spring/workout/processForm?name="
000000043: 200 15 L 31 W 234 Ch "client"
000000045: 200 15 L 31 W 235 Ch "cluster"
000000036: 200 15 L 31 W 233 Ch "chart"
000000035: 200 15 L 31 W 235 Ch "channel"
000000040: 200 15 L 31 W 235 Ch "classic"
000000039: 200 15 L 31 W 235 Ch "classes"
000000038: 200 15 L 31 W 233 Ch "class"
000000037: 200 15 L 31 W 232 Ch "chat"
000000031: 200 15 L 31 W 235 Ch "cgi-win"
000000033: 200 15 L 31 W 234 Ch "change"
```

Just as expected, I can see the values with less than 3 characters being rejected from here.

I then tested the simple XSS payload.

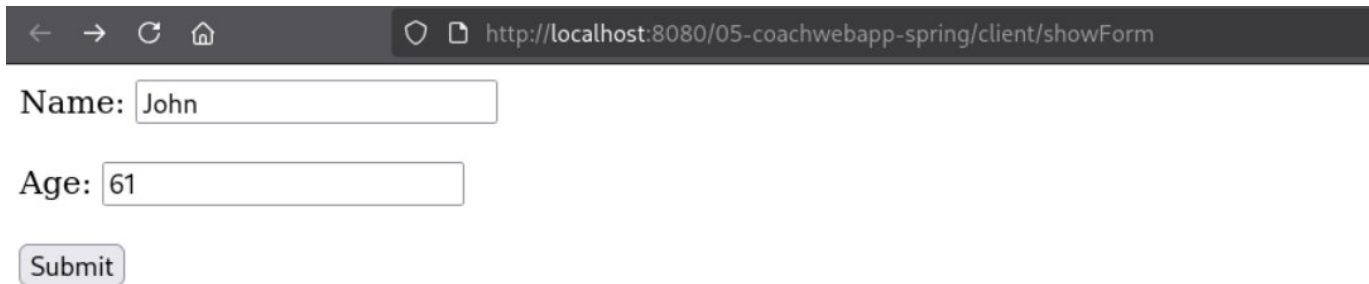


, please, run for 30 min

I updated the code as instructed. The `@Pattern` blocks values ending with ``<`, `>`, or `$``. If the validation fails, the form is shown again with an error message instead of crashing with a ``500`` error.

```
Client.java x ClientController.java CoachController.java
8 //import org.hibernate.validator.constraints
9
10 public class Client {
11
12     @NotNull(message = "is required")
13     @Size(min = 3, message="is required")
14     @Pattern(regexp = ".*[<>$]", message="incorrect format")
15     protected String name;
16
17     @RequestMapping("/processForm")
18     public String workout(
19         @Valid @ModelAttribute("client") Client client,
20         BindingResult validationErrors, Model model) {
21         //Input validation
22         if (validationErrors.hasErrors())
23             return "client-form";
24         // throw new IllegalArgumentException("Invalid Input");
25         //Logic when there is no error
26     }
27 }
```

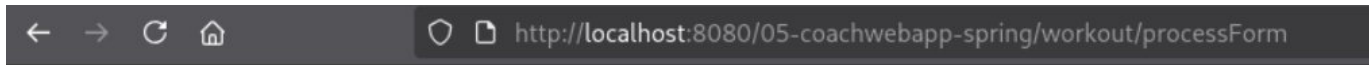

I then tested it with a value that I know will be accepted to ensure it works.



← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/client/showForm

Name:

Age:



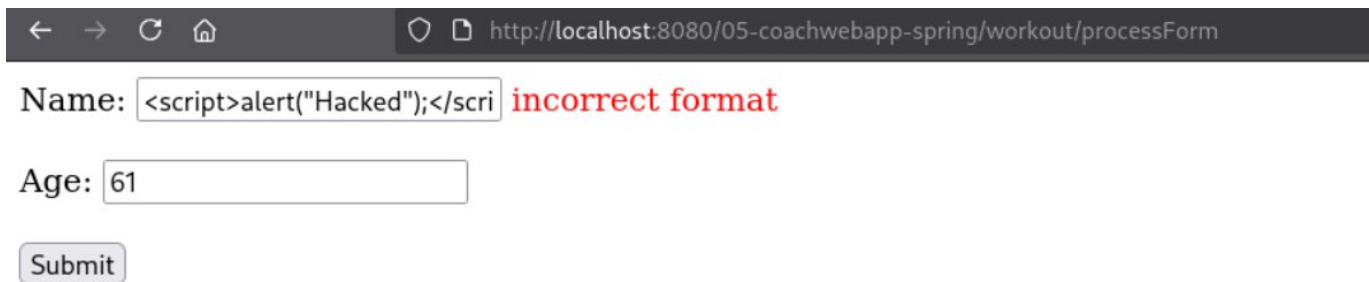
← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name: incorrect format

Age:

John, please, run for 30 min

Next, I tested the initial payload and it failed as intended.

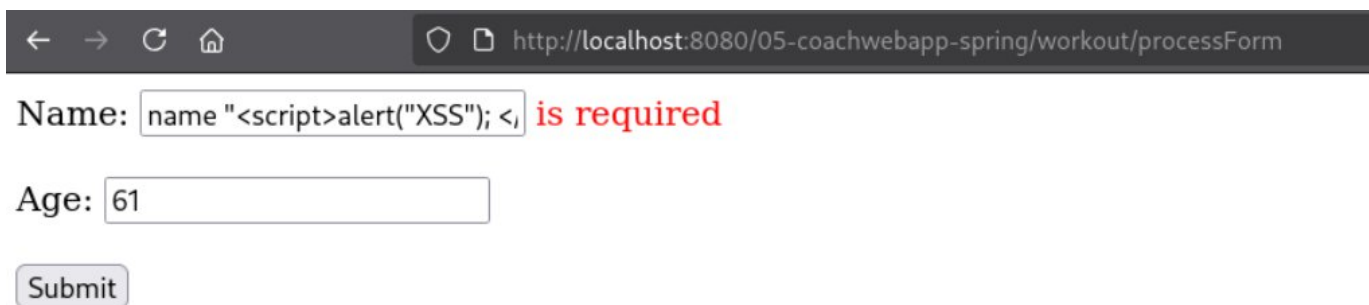


← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name: incorrect format

Age:

Then, I tried the first given payload.

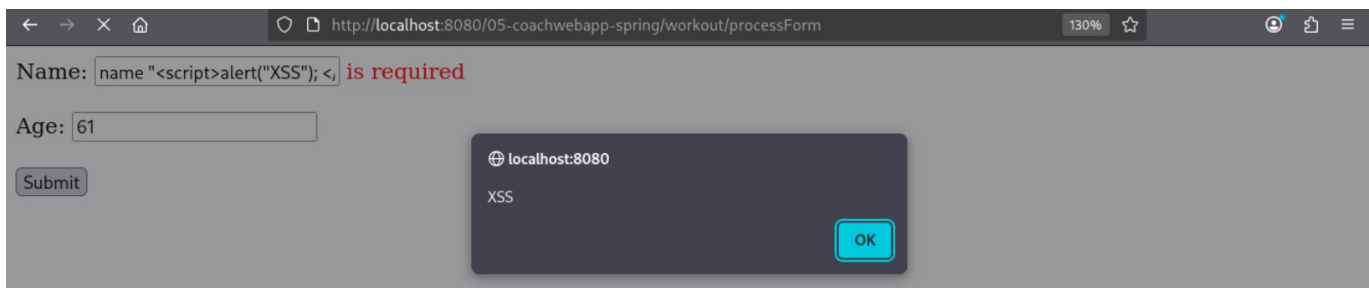


← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name: is required

Age:

And it worked as intended.



← → × 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm 130% ☆

Name: is required

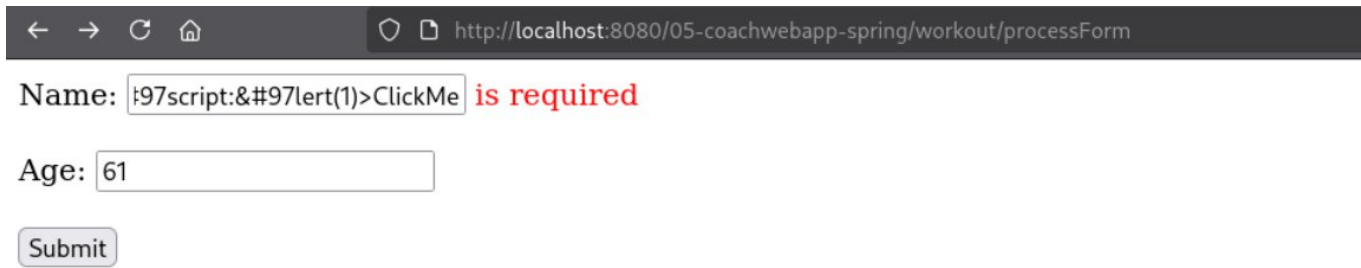
Age:

localhost:8080

XSS

OK

I then tested the second given payload.

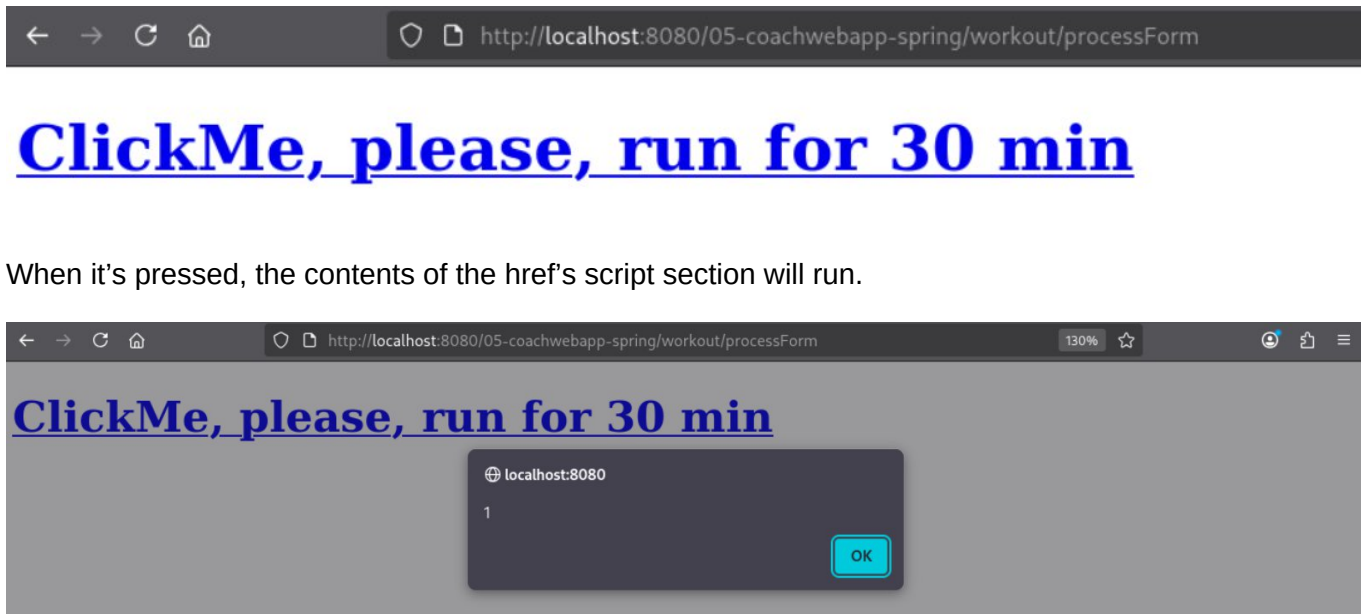


← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name: is required

Age:

That worked too. This text is displayed and it's a hyperlink (`<a>`).

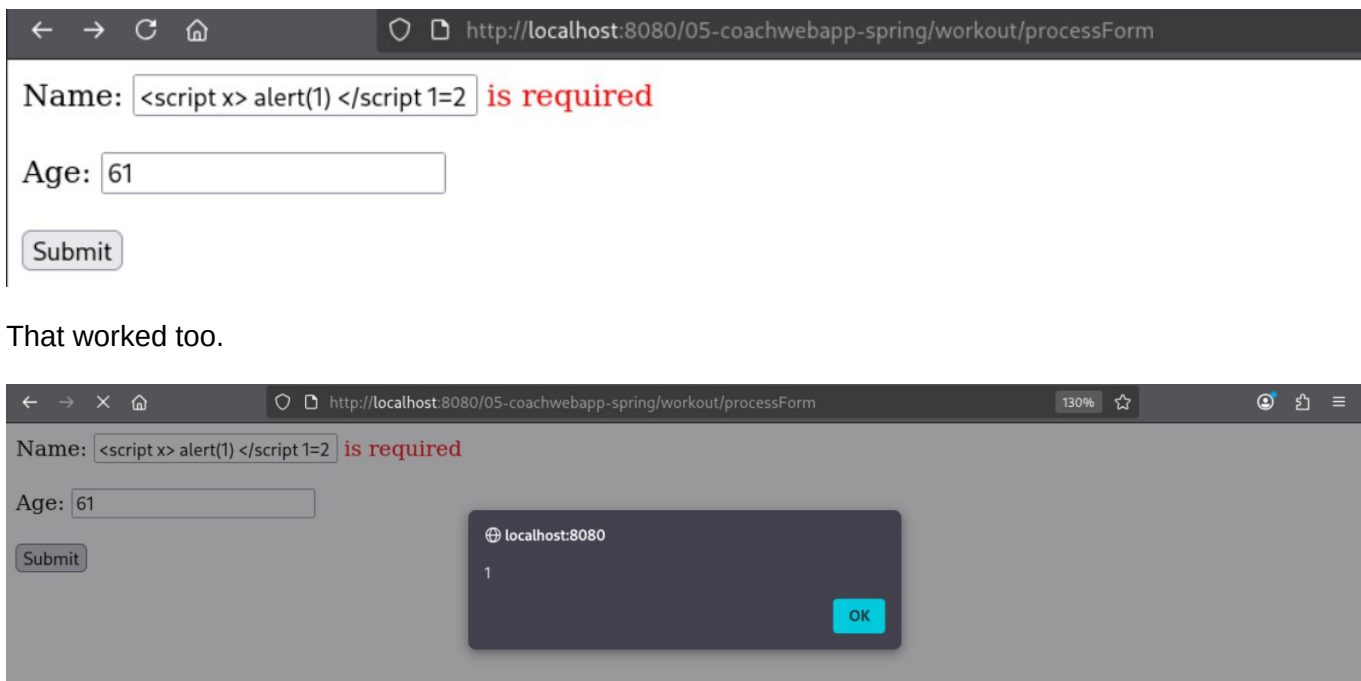


← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm 130% ☆

[ClickMe, please, run for 30 min](#)

localhost:8080
1

I then tested the third given payload.



← → × 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm 130% ☆

Name: is required

Age:

localhost:8080
1

The regex allows any characters, but checks that the final character is not <, >, or \$. This means it only validates the end of the input – not the full string.

Payload 1: name "<script>alert("XSS"); </script>"

This works because the input ends with a quote instead of >, so the regex allows it even though there is still a <script> tag inside.

Payload 2: <a ... href=javascript:alert(1)>ClickMe

This uses encoded characters to hide javascript:alert(1). The browser decodes it, but the regex does not properly detect it.

Payload 3: <script x> alert(1) </script 1=2

This ends with 2, so the regex allows it. The script tag is still inside the input, showing that only checking the last character is not enough.

f) Identify the type of validations that you performed in the 05-coachwebapp, (i.e. whitelisting or blacklisting?)

The validation technique used here is blacklisting. This is because it tries to block certain characters like <, >, and \$. The problem is that attackers can still bypass it by changing or encoding the input.

A better option here is whitelisting because then only allowed characters are accepted. For example, letters and spaces. This is safer because anything outside the allowed pattern is rejected.